Goal - define what it means for a table to be "redundancy good"

Example: bad

day	temp	city	mayor	madd
4/20	70°	Nfield	Zweifel	—
4/20	68%	Faribault	Musicant	—
4/19	32	Nfield	Zweifel	

⋮

FD city → mayor

functional dependency

Add mayor → mayor addr

Two FDs

city → mayor
mayor → mayor addr

} F = set of FDs

F^+ = closure of F = all FDs including F that can be derived from F

To get F^+, can repeatedly use Armstrong's Axioms

Let's define one version of "redundancy good."

Suppose that we have $XZ \to YZ$. Then which of the following are also true?

A $X \to Y$

B $XZ \to Y$ ✓

C Both

D Neither

If we know the values of X and Z, then we also know the values of Y and Z.

In particular $XZ \to Y$

$XZ \to Z$

author, title → publicationdate, title

X Z Y Z

I can't conclude

author → publication date

Armstrong: augmentation

$A \to B$

also $AZ \to BZ$

"Redundancy good" (one kind)

Boyce-Codd Normal Form (BCNF)

Relation R is in BCNF with respect to a set of FDs F (assuming all attrs in F are in R)

if, for all FDs $X \to A$ in F^+ either

(1) $A \subseteq X$ (trivial FD)

or

(2) X is a superkey for R.

Think about it this way.

FDs are dangerous.

If you're going to allow some FDs, only allow two types

Type 1 : trivial

$ABC \rightarrow A$

repeated

studnt id	addr	class year	dean
—	—	26	carlson
—	—	26	carlson
—	—	26	carlson
—	—	28	fleming

BUT if left side of the FD is a superkey for R, there's no repetition

(we said student id was a primary key)

student id → class year

id year addr dean

Suppose we have the relation $R(S, Y, A, P, D)$

ph

with FDs $S \to YAPD$ and $Y \to D$.

Suppose then we decompose it $R_1(S, Y, A, P)$ and $R_2(Y, D)$.

Are both R_1 and R_2 in BCNF?

R_2: YD

$Y \to D$ ✓ (Y superkey)

R_1: SYAP

$S \to YAP$ ← no D because D isn't in the reln

$S \to YAPD$ means

$S \to Y$ and

$S \to A$ and

$S \to P$ and

$S \to D$

so $S \to YAP$ is another FD

S is a superkey ✓

Typically, if not in BCNF, you think about decomposing into smaller relns to get to BCNF.

When you decompose a reln, typically two ways it might go wrong.

<u>Disaster #1</u>: Decompose into

R_1 : SYAD

R_2 : YP

I have lost the ability to connect ph# (P) back to a student

What happens when I join these together? (so I guess I join on Y)

→ you get lots of extraneous tuples that weren't in the original.

Lost data. BAD.

Critical feature: lossless-join decomposition.

- when you join back together, don't get get extraneous tuples.

If you decompose in a way that is not lossless-join, it's a disaster.

E.g.

~~Disaster~~ Mishap #2: when you fail to preserve all of the FDs across the decomposed relns.

Example:

Prof,	Hour,	Class
Dave	1a	CS334
Dave	2a	CS334
Layla	1a	CS111

FDs that specify:

$PH \to C$

$\underline{C \to P}$ [if you specify course, that tells you which prof

not a superkey

i.e. every course is always taught by the same prof]

Is this in BCNF?

& PHC

Might decompose as $C \to P$ one row for each C

HC CP

Is this a lossless-join decomp?

Yes, because each C is in just one row in CP.

But, we lost the ability to observe/enforce $PH \to C$

This is a decomp that is not dependency preserving.
(That's not wonderful)

BIG interesting result.

Give me a reln R.
Give me a set of FDs.
I can always find a decomp into a set of smaller relns that are all in BCNF via a lossless join decomp.
(But, can't always do it in a dependency preserving way)

What if I really care about both

① - lossless join

② - dep preserving?

I can always find a decomp into smaller relns, and get both ① and ② but I might not get BCNF but I can get to 3NF (third normal form)